

Regular: Context-Aware Page Ranking for DRAM Demotion in Hybrid Memory Systems

Anonymous Author(s)

ABSTRACT

DRAM–NVM hybrid memory systems address growing capacity demands by combining low-latency but capacity-limited DRAM with high-density yet slower non-volatile memory (NVM). Given limited DRAM capacity and NVM’s asymmetric read/write costs, hybrid memory systems must select an appropriate DRAM-resident page and demote it to NVM while retaining hot and write-sensitive pages in DRAM. Existing heuristics typically rely on coarse access signals, while learning-based policies can suffer from incomplete access context, misaligned objectives, or substantial online overhead. We propose **CAPD**, an efficient **Context-Aware Page-Demotion** policy that ranks DRAM-resident victim pages using recent access context and page states. CAPD is trained offline using replay-derived rankings based on future reuse, write sensitivity, and demotion cost, and is invoked only at demotion events. On PARSEC-derived traces, CAPD reduces weighted replay cost under DRAM capacity pressure, including a 10.83% reduction over the best classical baseline on the streamcluster pressure window.

1 INTRODUCTION

With the growth of data-intensive applications such as machine learning [6], graph analytics [29], and in-memory analytics [37], modern systems increasingly demand larger main-memory capacity and higher memory performance. However, DRAM scaling has become increasingly constrained [21], while non-volatile memory (NVM) provides higher density and lower cost per bit [12, 18, 23] but suffers from higher write latency [12, 36] and limited write endurance [13, 39]. Therefore, DRAM–NVM hybrid memory systems [3, 20, 24, 33], which combine low-latency DRAM with high-density NVM, have emerged as an important main-memory architecture.

In such systems, DRAM typically serves as a fast tier for hot and write-sensitive pages [1, 14, 24], while NVM provides a large-capacity tier for colder pages [15, 19, 25]. Because DRAM capacity is limited, the system must demote selected DRAM-resident pages to NVM under fast-tier pressure [11, 24, 27]. Demoting a soon-reused page can increase slow-tier accesses and access latency [8, 16, 24], or demoting a write-intensive page can increase NVM write traffic [14, 27] and accelerate device wear [13, 39].

Traditional replacement policies based on fixed heuristic rules, such as LRU, LFU, and CLOCK [2, 10], often struggle with dynamic access behavior [8, 14]. Learning-based approaches can improve adaptability by exploiting access history [17, 26, 28, 30] and program context [7], but many target cache hit rate or prefetching [7, 9, 17, 26, 28] rather than write sensitivity and the cost of demoting pages from DRAM to NVM, while longer-history models may incur nontrivial online overhead due to large history windows or complex inference [22, 28].

To address these limitations, we present CAPD, a lightweight context-aware page demotion policy for hybrid DRAM–NVM memory systems. CAPD encodes recent access records using physical addresses, program counters (PCs), and read/write flags, then lets each DRAM candidate extract candidate-specific context for demotion scoring.

In summary, this paper makes the following contributions:

- We formulate DRAM page demotion in hybrid memory as a cost-aware candidate-page ranking problem, enabling CAPD to select victims by comparing DRAM-resident candidates using recent access context and page-level states.
- We design a lightweight access-sequence encoder and candidate-aware scoring mechanism that generates demotion-priority scores from recent access context and page-level representations.
- We construct cost-aware reference rankings with offline replay and evaluate CAPD on real memory traces, showing cost reductions under DRAM capacity pressure and workload-dependent boundary cases.

2 BACKGROUND AND RELATED WORK

2.1 Cost-Aware Page Demotion

Beyond the basic DRAM–NVM tiering described above, the key policy question is which DRAM-resident page should be demoted under DRAM capacity pressure [5, 15, 24, 25]. This decision directly affects both performance and NVM write overhead: demoting soon-reused or write-intensive pages can increase slow-tier accesses [16, 24] and costly NVM writes [14, 27]. Demotion can also consume memory bandwidth and incur page-management overhead [31, 34, 35]. Therefore, DRAM-to-NVM demotion in hybrid memory

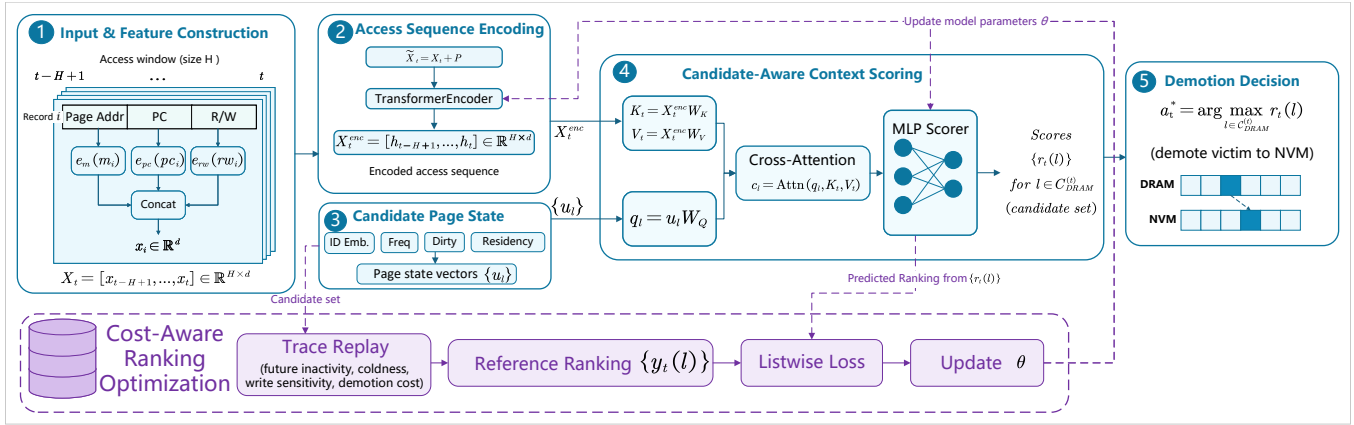


Figure 1: Overall framework of CAPD.

should not be reduced to a simple recency- or frequency-only victim-selection problem.

2.2 Context-Aware Candidate Ranking

Traditional policies such as LRU, LFU, and CLOCK are efficient, but they rely on fixed heuristics and reduce page behavior to simple signals such as recency or frequency [2, 10]. Two pages with similar recency or frequency can still have very different demotion costs: one may be read-only and cold [14], while another may be dirty, write-intensive [14], or likely to be reused soon [16, 24]. These differences matter in hybrid memory systems because write behavior and demotion cost directly affect the benefit of keeping a page in DRAM [14, 16].

Learning-based policies can use access history to improve adaptivity [17, 28, 30], but they must remain lightweight enough for demotion-time decisions. Modeling long histories or scoring each candidate against the full access sequence can introduce nontrivial online overhead [17, 22, 28]. At the same time, using only address sequences may miss useful signals from the program counter [7, 32] and read/write type [14].

These observations motivate a lightweight context-aware ranking formulation: at each demotion point, the policy should use recent access context and page-level states to rank the current DRAM candidates and demote the page with the highest demotion priority. CAPD follows this direction by learning a cost-aware candidate-page ranking policy for DRAM-to-NVM demotion.

3 DESIGN

3.1 Overall Framework

Figure 1 illustrates the overall framework of CAPD. CAPD treats DRAM page demotion as a context-aware candidate

ranking problem. It consists of four components: Multi-dimensional Feature Construction, Lightweight Access Sequence Encoding, Candidate-Aware Context Scoring and Demotion Decision, and Cost-Aware Ranking Optimization. It encodes recent accesses with a lightweight Transformer, lets each candidate page query the encoded sequence to obtain a candidate-specific context, and scores the context for demotion. Training uses offline trace replay to derive a cost-aware ranking objective.

3.2 Problem Definition

We consider a DRAM–NVM hybrid memory system, where DRAM offers low-latency access but has limited capacity, while NVM provides larger capacity but higher write overhead. When DRAM capacity pressure triggers a demotion decision at time t , the system must select one DRAM-resident victim page and demote it to NVM.

At each demotion point t , CAPD observes the recent access history $\mathcal{H}_t$ and a candidate set $C_{\text{DRAM}}^{(t)}$ of DRAM-resident pages eligible for demotion. CAPD assigns each candidate page l a demotion-priority score $r_t(l)$ and selects the highest-scoring page as the victim:

$$a_t^* = \arg \max_{l \in C_{\text{DRAM}}^{(t)}} r_t(l), \quad (1)$$

where $r_t(l)$ denotes the priority of demoting page l from DRAM to NVM at time t .

3.3 Multi-dimensional Feature Construction

CAPD constructs multi-dimensional access features from memory-access records. Each record contains a page identifier m_t derived from the physical address, a program counter identifier pc_t , and a read/write flag rw_t , where $rw_t = 0$ for

reads and $rw_t = 1$ for writes. These fields capture page-level locality, program context, and access type, respectively. CAPD embeds and concatenates these fields as:

$$x_t = e_m(m_t) \parallel e_{pc}(pc_t) \parallel e_{rw}(rw_t) \in \mathbb{R}^d, \quad (2)$$

where $\parallel$ denotes concatenation and $d = d_m + d_{pc} + d_{rw}$. At a demotion point t , CAPD constructs a historical access window from the most recent H access representations:

$$X_t = [x_{t-H+1}, \dots, x_t]^\top \in \mathbb{R}^{H \times d}, \quad (3)$$

where H denotes the sliding-window length.

3.4 Lightweight Access Sequence Encoding

At each demotion point t , CAPD treats the historical window X_t as an ordered sequence and adds positional encodings:

$$\tilde{X}_t = X_t + P, \quad (4)$$

where $P \in \mathbb{R}^{H \times d}$ denotes the positional encoding.

A lightweight single-layer Transformer encoder then produces contextualized access representations:

$$X_t^{enc} = \text{TEnc}_1(\tilde{X}_t) = [h_{t-H+1}, h_{t-H+2}, \dots, h_t]^\top \in \mathbb{R}^{H \times d}. \quad (5)$$

Here, each h_i denotes a contextualized representation.

3.5 Context-Aware Candidate Page Ranking and Demotion Decision

When a DRAM page demotion decision is triggered at time t , CAPD constructs a page-level representation u_l for each candidate page $l \in C_{\text{DRAM}}^{(t)}$ from its page identifier embedding and lightweight state features, including recent frequency, dirty status, and residency duration. Rather than compressing the access history into a single shared context for all candidates, CAPD uses each candidate page representation as a query to extract a candidate-specific context from the encoded access sequence. CAPD projects the candidate representation into a query and the encoded access sequence into keys and values:

$$q_l = u_l W_Q, \quad K_t = X_t^{enc} W_K, \quad V_t = X_t^{enc} W_V. \quad (6)$$

CAPD then computes the candidate-specific context of page l through cross-attention:

$$c_l = \text{softmax}\left(\frac{q_l K_t^\top}{\sqrt{d_a}}\right) V_t, \quad (7)$$

where d_a denotes the attention dimension. The resulting vector c_l is the candidate-specific context for page l .

CAPD scores this context with a MLP:

$$r_t(l) = \text{MLP}(c_l), \quad (8)$$

where $r_t(l)$ denotes the DRAM-to-NVM demotion-priority score.

Ultimately, CAPD demotes the highest-scoring page:

$$a_t^* = \arg \max_{l \in C_{\text{DRAM}}^{(t)}} r_t(l). \quad (9)$$

3.6 Cost-Aware Ranking Optimization

During training, CAPD uses offline trace replay to assign each candidate page $l \in C_{\text{DRAM}}^{(t)}$ a cost-aware demotion relevance score:

$$y_t(l) = \lambda_1 \hat{d}_t(l) + \lambda_2 \hat{q}_t(l) - \lambda_3 \hat{w}_t(l) - \lambda_4 \hat{c}_t(l), \quad (10)$$

The four terms are computed from the future lookahead window of length $L = 256$ after demotion point t . Let $\delta_t(l)$ be the distance from t to the next access to page l in this window. If l is not accessed again within the window, we set $\hat{d}_t(l) = 1$; otherwise $\hat{d}_t(l) = \delta_t(l)/L$. Thus, future inactivity captures the distance to the next reuse. Coldness captures a different signal: the total future access frequency. If $f_t(l)$ is the number of future accesses to l in the lookahead window, then

$$\hat{q}_t(l) = 1 - \min\left(\frac{f_t(l)}{L}, 1\right). \quad (11)$$

A page can therefore have a long next-reuse distance but still not be cold if it is accessed repeatedly later in the window.

Write sensitivity measures future write pressure. Let $f_t^W(l)$ be the number of future writes to page l in the same lookahead window. We define

$$\hat{w}_t(l) = \min\left(\frac{f_t^W(l)}{L}, 1\right). \quad (12)$$

Higher write sensitivity lowers the demotion relevance because demoting a write-active page to NVM is undesirable. The demotion-cost term $\hat{c}_t(l)$ is a normalized migration-cost proxy. In the current 4KB-page trace replay, valid candidates have the same page size, so this term is not a measured device-latency model; it is kept as a page-level cost hook for future variable-cost demotion models.

The default weights are $(\lambda_1, \lambda_2, \lambda_3, \lambda_4) = (1, 1, 4, 2)$. All four components are bounded in $[0, 1]$ by construction. After computing $y_t(l)$, CAPD applies min-max normalization over the valid candidates in $C_{\text{DRAM}}^{(t)}$ before computing the ranking loss:

$$\tilde{y}_t(l) = \frac{y_t(l) - \min_{j \in C_{\text{DRAM}}^{(t)}} y_t(j)}{\max_{j \in C_{\text{DRAM}}^{(t)}} y_t(j) - \min_{j \in C_{\text{DRAM}}^{(t)}} y_t(j) + \epsilon}. \quad (13)$$

A larger $\tilde{y}_t(l)$ indicates a better demotion candidate.

CAPD normalizes the predicted demotion-priority scores over the same candidate set:

$$\pi_\theta(a_t = l \mid s_t) = \frac{\exp(r_t(l))}{\sum_{j \in C_{\text{DRAM}}^{(t)}} \exp(r_t(j))}. \quad (14)$$

Here, $\pi_\theta(a_t = l \mid s_t)$ denotes the probability of selecting page l as the victim. CAPD optimizes the candidate ordering with a listwise approximate-NDCG loss:

$$\mathcal{L}_{rank}(s_t, \pi_\theta) = -\frac{DCG(s_t, \pi_\theta)}{IDCG(s_t)}. \quad (15)$$

The DCG term uses $\tilde{y}_t(l)$ as the relevance and a differentiable rank position induced by the predicted scores. For candidate l_i , the approximate 1-based rank position is

$$\rho_t(l_i) = 1 + \sum_{j \neq i} \sigma(\alpha(r_t(l_j) - r_t(l_i))), \quad (16)$$

where $\sigma(\cdot)$ is the sigmoid function and $\alpha = 10$. The gain is $\exp(\tilde{y}_t(l)) - 1$, and the discount is $\log(1 + \rho_t(l))$. This training objective encourages CAPD to rank pages with higher cost-aware demotion relevance closer to the top.

3.7 Complexity and System Overhead

For each demotion decision, CAPD encodes an access window of length H , computes candidate-specific contexts through cross-attention, and scores all pages in $C_{\text{DRAM}}^{(t)}$. With feature dimension d and attention dimension d_a , the per-decision inference cost is

$$O(H^2d + Hd^2 + |C_{\text{DRAM}}^{(t)}|Hd + |C_{\text{DRAM}}^{(t)}|d^2), \quad (17)$$

where the first two terms come from the single-layer Transformer encoder, the third term comes from candidate-aware cross-attention over the access window, and the last term comes from lightweight page scoring. Positional encoding adds only $O(Hd)$ lower-order cost.

4 EVALUATION

4.1 Experimental Setup

We evaluate CAPD using trace replay on PARSEC-derived memory traces. We collect user-space memory references with DynamoRIO/drmemtrace and convert the drmemtrace view output into PC, Address, RW records. The recorded addresses are virtual addresses from the traced process. We align each address to a 4KB page with `page_shift=12` and use the resulting virtual page identifier in replay; the prototype does not reconstruct physical addresses.

For the standard 1M-trace experiments, each workload contains 1,000,000 memory references and is split chronologically into 800K training, 100K validation, and 100K test references. All reported standard-split results use the held-out test window. For pressure-window experiments, we select test windows before CAPD evaluation using only LRU replacement pressure under a 16-page DRAM, not CAPD performance. The selected streamcluster pressure window uses references 0–270K for training, 270K–370K for validation, and 370K–570K for testing. The selected dedup pressure

window uses references 0–700K for training, 700K–800K for validation, and 800K–1M for testing.

Unless otherwise stated, the DRAM tier holds 16 pages, the history window contains 10 accesses, the eviction candidate set contains 8 pages, and the prediction lookahead is 256 accesses. At each DRAM miss with a full DRAM tier, the candidate set is constructed from the 8 pages at the LRU tail of the current DRAM-resident set.

CAPD uses 8-dimensional page-address embeddings, 8-dimensional PC embeddings, and 2-dimensional read/write embeddings, giving an 18-dimensional access representation. The sequence encoder is a one-layer Transformer with 2 attention heads and a 72-dimensional feed-forward layer. Each candidate page is represented by an 8-dimensional page embedding and four state features: recent frequency, dirty status, normalized residency duration, and normalized LRU-tail rank. The candidate scorer uses 2-head cross-attention from candidate pages to the encoded access sequence, followed by an MLP with hidden dimensions 36 and 18. We train for 10 epochs with batch size 32.

We compare CAPD against four classical replacement policies, LRU, Random, LFU, and CLOCK, and two learned trace-replay baselines. Kleio-lite follows Kleio’s neural page-hotness prediction and page-prioritization logic [4]. PatternS-lite follows PatternS’s page-pattern clustering and group-level hotness-prediction logic [38]. These baselines are replay-compatible adaptations rather than full-system reproductions.

We use a weighted cost metric:

$$C = N_{\text{hit}} + 2N_{\text{nvm_read}} + 8N_{\text{nvm_write}} + 10N_{\text{demotion}}.$$

This metric charges NVM writes and demotions more heavily than DRAM hits, matching the asymmetric cost structure of hybrid DRAM–NVM memory. Lower cost is better.

Unless otherwise stated, Figure ?? and Table 1 report the result of a pre-specified default training seed, 3136859. This seed was fixed before inspecting the test results and is used consistently for the main replay comparison. Section ?? separately reports three-seed stability results to show which conclusions are robust to training randomness.

4.2 Main Results

Figure 2 summarizes the main replay results. CAPD improves the weighted cost on workloads where replacement decisions expose a real tradeoff between hit rate, writes, and demotions. On streamcluster-pressure, CAPD reduces normalized cost by 10.83% over the best classical baseline and by 18.93% over the best learned baseline. On blackscholes, the default-seed run slightly improves over the best baseline, but Section ?? shows that this small gain is seed-sensitive. We therefore treat blackscholes as a near-tie rather than as primary evidence for CAPD’s benefit.

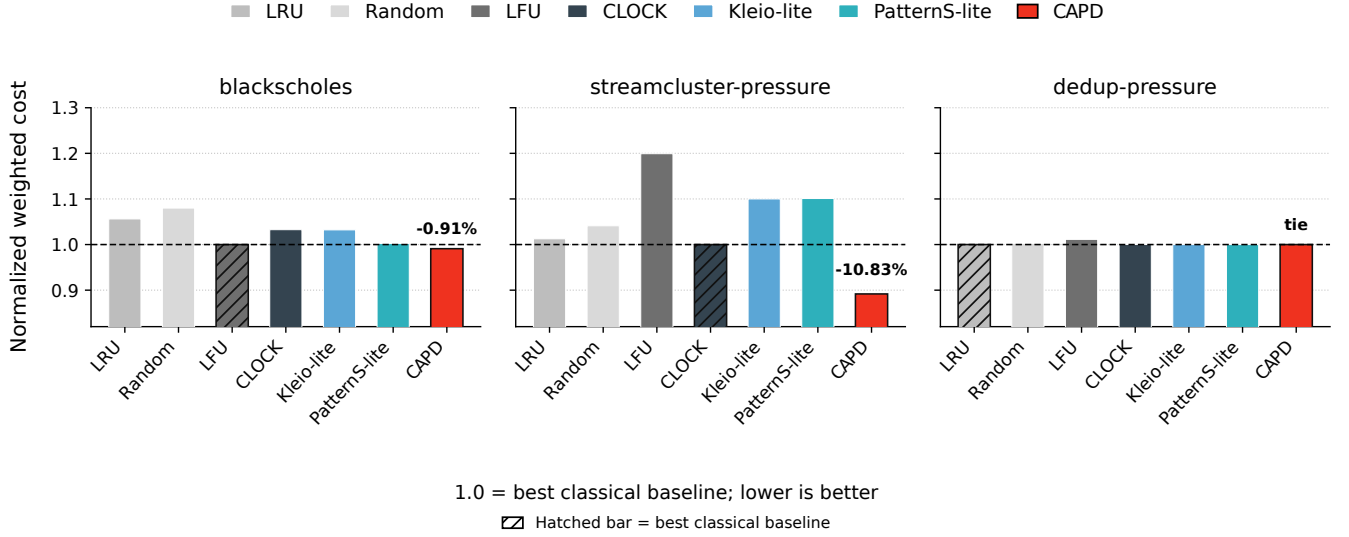


Figure 2: Overall weighted-cost comparison under the pre-specified default seed 3136859. Costs are normalized to the best classical baseline in each workload; lower is better. The hatched bar marks the best classical baseline.

Table 1: Main replay metrics under the pre-specified default seed 3136859. Weighted cost, NVM writes, and migrations are lower-is-better; hit rate is higher-is-better.

Workload	Policy	Weighted cost	Hit rate (%)	NVM reads	NVM writes	Demotions
blackscholes	LRU	112,958	98.89	970	144	1,098
	Random	115,505	98.66	1,200	145	1,329
	LFU	106,952	99.47	305	221	510
	CLOCK	110,437	99.09	798	107	889
	Kleio-lite	110,390	99.04	956	2	942
	PatternS-lite	107,207	99.36	594	49	627
	CAPD	105,983	99.46	509	32	525
streamcluster-pressure	LRU	305,562	95.35	8,707	585	9,276
	Random	314,183	95.01	9,194	777	9,955
	LFU	361,877	92.84	13,587	740	14,311
	CLOCK	301,767	95.52	8,379	574	8,937
	Kleio-lite	331,941	94.19	10,932	697	11,613
	PatternS-lite	332,226	94.15	11,063	629	11,676
	CAPD	269,095	97.00	5,449	548	5,981
dedup-pressure	LRU	201,567	99.95	4	99	87
	Random	202,076	99.93	41	105	130
	LFU	203,845	99.88	4	233	221
	CLOCK	201,584	99.95	4	100	88
	Kleio-lite	201,567	99.95	4	99	87
	PatternS-lite	201,567	99.95	4	99	87
	CAPD	201,567	99.95	4	99	87

Table 1 gives the same comparison in absolute weighted cost. The strongest result is on streamcluster-pressure, where CAPD reduces demotions from 8937 under CLOCK to 5981 while also increasing the hit rate from 95.52% to 97.00%. This shows that CAPD is not merely reducing writes by sacrificing locality; it learns eviction choices that jointly reduce NVM reads, NVM writes, and demotions. On blackscholes, CAPD is close to LFU in hit rate but writes substantially fewer NVM pages, leading to a small overall cost reduction. On

Table 2: Capacity sensitivity on streamcluster-pressure.

DRAM pages	Best rule	CAPD	Δ	CAPD demotions
8	369999	332660	-10.09%	11738
16	301767	269095	-10.83%	5981
32	216577	217348	+0.36%	1566

dedup-pressure, almost all competitive policies converge to the same behavior, leaving little room for improvement.

4.3 Capacity Sensitivity

We next vary the DRAM capacity on streamcluster-pressure, the workload with the clearest memory-pressure behavior. This experiment tests whether CAPD only wins at a single cache size or remains useful across pressure levels.

Table 2 shows that CAPD is most useful when DRAM pressure is high. With 8 and 16 DRAM pages, it reduces weighted cost by about 10% relative to the best rule-based policy. With 32 pages, the miss stream shrinks substantially and replacement decisions become less consequential; CAPD then converges to near parity with the best classical baseline. This behavior is expected for a learned eviction policy: its advantage appears when the cache is small enough that choosing the wrong victim causes repeated NVM accesses or demotions.

4.4 Stability and Robustness

Because CAPD is trained, we check whether the main result depends on a single training seed. The main Figure ?? and Table 1 use the pre-specified default seed 3136859. We then retrain CAPD with seeds 3136859, 42, and 2026 while reusing the same deterministic rule-based baselines.

On streamcluster-pressure, all three seeds beat the best classical baseline: the cost reduction ranges from 10.69% to 11.71%, with mean 11.08% and standard deviation 0.45%. This supports streamcluster-pressure as the main positive result. On blackscholes, the default seed improves cost by 0.91%, but the three-seed mean is +0.24% with standard deviation 1.71%; one seed is 2.66% worse than LFU. We therefore interpret blackscholes as a near-tie rather than a robust win. On canneal, all three seeds are worse than LRU, with mean delta +16.81%, confirming it as a stable negative boundary case.

We also reweight the dominant pressure case under alternative cost models. On streamcluster-pressure, CAPD remains better than the best classical baseline under mild, write-heavy, and demotion-heavy cost settings, with reductions from 6.98% to 15.91%. This suggests that the result is not an artifact of a single hand-picked write or demotion weight.

4.5 Discussion

The results support two observations. First, the complete CAPD framework can improve DRAM-to-NVM demotion decisions under sustained capacity pressure. On streamcluster-pressure, CAPD improves the hit rate while reducing NVM reads, NVM writes, and demotions, and its weighted-cost reduction remains stable across training seeds and alternative cost settings.

Second, CAPD's benefit is workload-dependent. CAPD remains near parity with the strongest baselines on blackscholes and dedup-pressure, while on canneal it

increases weighted replay cost by 15.32% relative to the best classical baseline. This negative case shows that the learned ranking policy does not uniformly outperform classical policies across all trace regimes. Overall, the results suggest that context-aware candidate ranking is most useful when demotion choices materially affect future slow-tier accesses and page-movement cost.

5 CONCLUSION

This paper presented CAPD, a context-aware page-demotion policy for DRAM–NVM hybrid memory systems. CAPD formulates DRAM victim selection as a candidate-page ranking problem: it encodes recent memory accesses and uses each candidate's page representation to derive candidate-specific context for demotion scoring. Offline trace replay provides cost-aware reference rankings, while online inference is invoked only when DRAM capacity pressure triggers a demotion event. On PARSEC-derived traces, CAPD achieves its clearest benefit under sustained pressure, reducing weighted replay cost by 10.83% on streamcluster-pressure. It remains near parity on blackscholes and dedup-pressure, but underperforms on canneal, demonstrating that its benefit is workload-dependent. These results establish context-aware candidate ranking as a promising direction for DRAM-to-NVM demotion while motivating broader workload evaluation and runtime integration.

REFERENCES

- [1] Neha Agarwal and Thomas F Wenisch. 2017. Thermostat: Application-transparent page management for two-tiered main memory. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems*. 631–644.
- [2] Laszlo A. Belady. 1966. A study of replacement algorithms for a virtual-storage computer. *IBM Systems journal* 5, 2 (1966), 78–101.
- [3] Gaurav Dhiman, Raid Ayoub, and Tajana Rosing. 2009. PDRAM: A hybrid PRAM and DRAM main memory system. In *Proceedings of the 46th Annual Design Automation Conference*. 664–669.
- [4] Thaleia Dimitra Doudali, Sergey Blagodurov, Abhinav Vishnu, Sudhanva Gurumurthi, and Ada Gavrilovska. 2019. Kleio: A Hybrid Memory Page Scheduler with Machine Intelligence. In *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing*. 37–48. <https://doi.org/10.1145/3307681.3325398>
- [5] Subramanya R Dullloor, Amitabha Roy, Zhenguang Zhao, Narayanan Sundaram, Nadathur Satish, Rajesh Sankaran, Jeff Jackson, and Karsten Schwan. 2016. Data tiering in heterogeneous memory systems. In *Proceedings of the Eleventh European Conference on Computer Systems*. 15:1–15:16.
- [6] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W Mahoney, and Kurt Keutzer. 2024. Ai and memory wall. *IEEE Micro* 44, 3 (2024), 33–39.
- [7] Milad Hashemi, Kevin Swersky, Jamie Smith, Grant Ayers, Heiner Litz, Jichuan Chang, Christos Kozyrakis, and Parthasarathy Ranganathan. 2018. Learning memory access patterns. In *International Conference on Machine Learning*. PMLR, 1919–1928.
- [8] Taekyung Heo, Yang Wang, Wei Cui, Jaehyuk Huh, and Lintao Zhang. 2022. Adaptive Page Migration Policy with Huge Pages in Tiered

- Memory Systems. *IEEE Trans. Comput.* 71, 1 (2022), 53–68. <https://doi.org/10.1109/TC.2020.3036686>
- [9] Xinyue Hu, Eman Ramadan, Wei Ye, Feng Tian, and Zhi-Li Zhang. 2022. Raven: belady-guided, predictive (deep) learning for in-memory and content caching. In *Proceedings of the 18th International Conference on emerging Networking EXperiments and Technologies*. 72–90.
- [10] Song Jiang, Feng Chen, and Xiaodong Zhang. 2005. CLOCK-Pro: An Effective Improvement of the CLOCK Replacement. In *USENIX ATC, General Track*. 323–336.
- [11] Sudarsun Kannan, Ada Gavrilovska, Vishal Gupta, and Karsten Schwan. 2017. Heteroos: Os design for heterogeneous memory management in datacenter. In *Proceedings of the 44th Annual International Symposium on Computer Architecture*. 521–534.
- [12] Benjamin C. Lee, Engin Ipek, Onur Mutlu, and Doug Burger. 2009. Architecting Phase Change Memory as a Scalable DRAM Alternative. In *Proceedings of the 36th Annual International Symposium on Computer Architecture (ISCA)*.
- [13] Benjamin C Lee, Ping Zhou, Jun Yang, Youtao Zhang, Bo Zhao, Engin Ipek, Onur Mutlu, and Doug Burger. 2010. Phase-change technology and the future of main memory. *IEEE micro* 30, 1 (2010), 143.
- [14] Soyoon Lee, Hyokyung Bahn, and Sam H Noh. 2014. CLOCK-DWF: A write-history-aware page replacement algorithm for hybrid PCM and DRAM memory architectures. *IEEE Trans. Comput.* 63, 9 (2014), 2187–2200.
- [15] Taehyung Lee, Sumit Kumar Monga, Changwoo Min, and Young Ik Eom. 2023. Memtis: Efficient memory tiering with dynamic page classification and page size determination. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 17–34.
- [16] Yang Li, Saugata Ghose, Jongmoo Choi, Jin Sun, Hui Wang, and Onur Mutlu. 2017. Utility-based hybrid memory management. In *2017 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 152–165.
- [17] Evan Liu, Milad Hashemi, Kevin Swersky, Parthasarathy Ranganathan, and Junwhan Ahn. 2020. An imitation learning approach for cache replacement. In *International Conference on Machine Learning*. PMLR, 6237–6247.
- [18] Hai-Kun Liu, Di Chen, Hai Jin, Xiao-Fei Liao, Bingsheng He, Kan Hu, and Yu Zhang. 2021. A survey of non-volatile main memory technologies: State-of-the-arts, practices, and future directions. *Journal of Computer Science and Technology* 36, 1 (2021), 4–32.
- [19] Hasan Al Maruf, Hao Wang, Abhishek Dhanotia, Johannes Weiner, Niket Agarwal, Pallab Bhattacharya, Chris Petersen, Mosharaf Chowdhury, Shobhit Kanaujia, and Prakash Chauhan. 2023. Tpp: Transparent page placement for cxl-enabled tiered-memory. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 742–755.
- [20] Jeffrey C. Mogul, Eduardo Argollo, Mehul M. Shah, and Paolo Faraboschi. 2009. Operating System Support for NVM+DRAM Hybrid Main Memory. In *Proceedings of the 12th USENIX Workshop on Hot Topics in Operating Systems (HotOS)*.
- [21] Onur Mutlu. 2013. Memory scaling: A systems architecture perspective. In *2013 5th IEEE International Memory Workshop*. 21–25. <https://doi.org/10.1109/IMW.2013.6582088>
- [22] Songwen Pei, Jianan Li, Yihuan Qian, Jie Tang, and Jean-Luc Gaudiot. 2022. TransMigrator: A Transformer-Based Predictive Page Migration Mechanism for Heterogeneous Memory. In *Network and Parallel Computing: 19th IFIP WG 10.3 International Conference, NPC 2022, Jinan, China, September 24–25, 2022, Proceedings* (Jinan, China). Springer-Verlag, Berlin, Heidelberg, 180–191. https://doi.org/10.1007/978-3-031-21395-3_17
- [23] Moinuddin K. Qureshi, Vijayalakshmi Srinivasan, and Jude A. Rivers. 2009. Scalable High Performance Main Memory System Using Phase-Change Memory Technology. In *Proceedings of the 36th Annual International Symposium on Computer Architecture (ISCA)*.
- [24] Luiz E Ramos, Eugene Gorbatov, and Ricardo Bianchini. 2011. Page placement in hybrid memory systems. In *Proceedings of the international conference on Supercomputing*. 85–95.
- [25] Amanda Raybuck, Tim Stamler, Wei Zhang, Mattan Erez, and Simon Peter. 2021. Hemem: Scalable tiered memory management for big data applications and real nvm. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. 392–407.
- [26] Liana V Rodriguez, Farzana Beente Yusuf, Steven Lyons, Eysler Paz, Raju Rangaswami, Jason Liu, Ming Zhao, and Giri Narasimhan. 2021. Learning Cache Replacement with CACHEUS. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*. 341–354.
- [27] Reza Salkhordeh and Hossein Asadi. 2016. An operating system level data migration scheme in hybrid DRAM-NVM memory architecture. In *2016 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 936–941.
- [28] Zhan Shi, Xiangru Huang, Akanksha Jain, and Calvin Lin. 2019. Applying deep learning to the cache replacement problem. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 413–425.
- [29] Julian Shun and Guy E. Blelloch. 2013. Ligma: A Lightweight Graph Processing Framework for Shared Memory. In *Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '13)*. ACM, 135–146. <https://doi.org/10.1145/2442516.2442530>
- [30] Giuseppe Vietri, Liana V Rodriguez, Wendy A Martinez, Steven Lyons, Jason Liu, Raju Rangaswami, Ming Zhao, and Giri Narasimhan. 2018. Driving Cache Replacement with ML-Based LeCaR. In *10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 18)*.
- [31] Xiaoyuan Wang, Haikun Liu, Xiaofei Liao, Ji Chen, Hai Jin, Yu Zhang, Long Zheng, Bingsheng He, and Song Jiang. 2019. Supporting Superpages and Lightweight Page Migration in Hybrid Memory Systems. *ACM Transactions on Architecture and Code Optimization* 16, 2 (2019), 11:1–11:26.
- [32] Carole-Jean Wu, Aamer Jaleel, Will Hasenplaugh, Margaret Martonosi, Simon C Steely Jr, and Joel Emer. 2011. SHiP: Signature-based hit predictor for high performance caching. In *Proceedings of the 44th Annual IEEE/ACM international symposium on microarchitecture*. 430–441.
- [33] Kai Wu, Yingchao Huang, and Dong Li. 2017. Unimem: Runtime Data Management on Non-Volatile Memory-Based Heterogeneous Main Memory. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. ACM, Article 58, 14 pages. <https://doi.org/10.1145/3126908.3126923>
- [34] Lingfeng Xiang, Zhen Lin, Weishu Deng, Hui Lu, Jia Rao, Yifan Yuan, and Ren Wang. 2024. Nomad: Non-Exclusive Memory Tiering via Transactional Page Migration. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 19–35.
- [35] Zi Yan, Daniel Lustig, David Nellans, and Abhishek Bhattacharjee. 2019. Nimble page management for tiered memory systems. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. 331–345.
- [36] Jian Yang, Juno Kim, Morteza Hoseinzadeh, Joseph Izraelevitz, and Steve Swanson. 2020. An empirical guide to the behavior and use of scalable persistent memory. In *18th USENIX Conference on File and Storage Technologies (FAST 20)*. 169–182.
- [37] Matei Zaharia, Mosharaf Chowdhury, Rathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. 2012. Resilient Distributed Datasets: A Fault-Tolerant

743	Abstraction for In-Memory Cluster Computing. In <i>9th USENIX symposium on networked systems design and implementation (NSDI 12)</i> . 15–28.	Page Pattern Recognition. <i>Journal of Systems Architecture</i> 153 (2024), 103178. https://doi.org/10.1016/j.sysarc.2024.103178	796
744			797
745			798
746	[38] Yanjie Zhen, Weining Chen, Wei Gao, Ju Ren, Kang Chen, and Yu Chen. 2024. PatternS: An Intelligent Hybrid Memory Scheduler Driven by	[39] Ping Zhou, Bo Zhao, Jun Yang, and Youtao Zhang. 2009. A durable and energy efficient main memory using phase change memory technology. <i>ACM SIGARCH computer architecture news</i> 37, 3 (2009), 14–23.	799
747			800
748			801
749			802
750			803
751			804
752			805
753			806
754			807
755			808
756			809
757			810
758			811
759			812
760			813
761			814
762			815
763			816
764			817
765			818
766			819
767			820
768			821
769			822
770			823
771			824
772			825
773			826
774			827
775			828
776			829
777			830
778			831
779			832
780			833
781			834
782			835
783			836
784			837
785			838
786			839
787			840
788			841
789			842
790			843
791			844
792			845
793			846
794			847
795			848